



Programação Avançada 2025-26

[16] Padrões de Software | Strategy

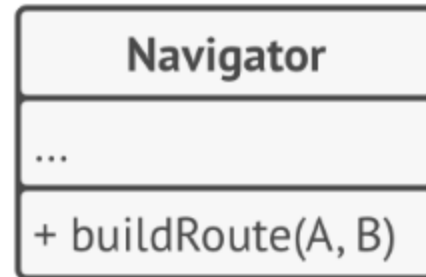
Bruno Silva, Patrícia Macedo

Sumário

- Padrão de desenho **Strategy**
 - Enquadramento;
 - Motivação;
 - Solução Proposta (pelo padrão);
 - Exemplo de Aplicação;
 - Exercícios.
 - Prós e contras.

Strategy | Exemplo de Enquadramento

Temos uma classe `Navigator` que será responsável por calcular rotas entre dois locais A e B.



A rota pode ser calculada através de diferentes critérios:

- De carro;
- Transportes públicos;
- A pé;

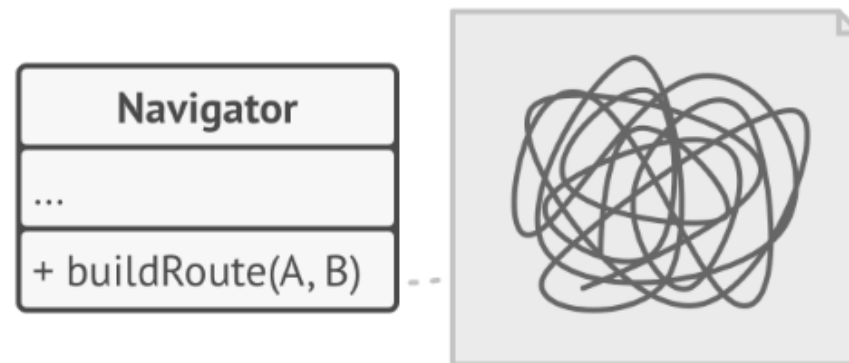
Strategy | Exemplo de Enquadramento

À partida, isto pode ser resolvido de três formas:

1. A classe `Navigator` possui um atributo `criteria` mutável (e.g., `enum Criteria {CAR, PUBLIC_TRANSPORT, WALK}`) e o método `buildRoute(A,B)` que tem um *comportamento* que se ajusta consoante o valor deste atributo.
 - O *cliente* altera o critério desejado e invoca o método.
2. Diferentes métodos para os diferentes critérios, e.g., `buildRouteByCar(A,B)`, `buildRouteByPublicTransportation(A,B)`, etc.
 - O *cliente* invoca o método associado ao critério desejado.
3. Através de herança/polimorfismo: e.g., sub-classes `NavigatorByCar`, `NavigatorByPublicTransportation`, etc.
 - O *cliente* instancia a classe apropriada e invoca o método.

Strategy | Exemplo de Enquadramento 🤔

👉 As "soluções" 1 e 2 implicam que, sempre que seja adicionado um novo critério, e.g., BICICLETA, seja necessário alterar a própria classe `Navigator`. Ao longo do tempo esta classe irá ficar difícil de manter.



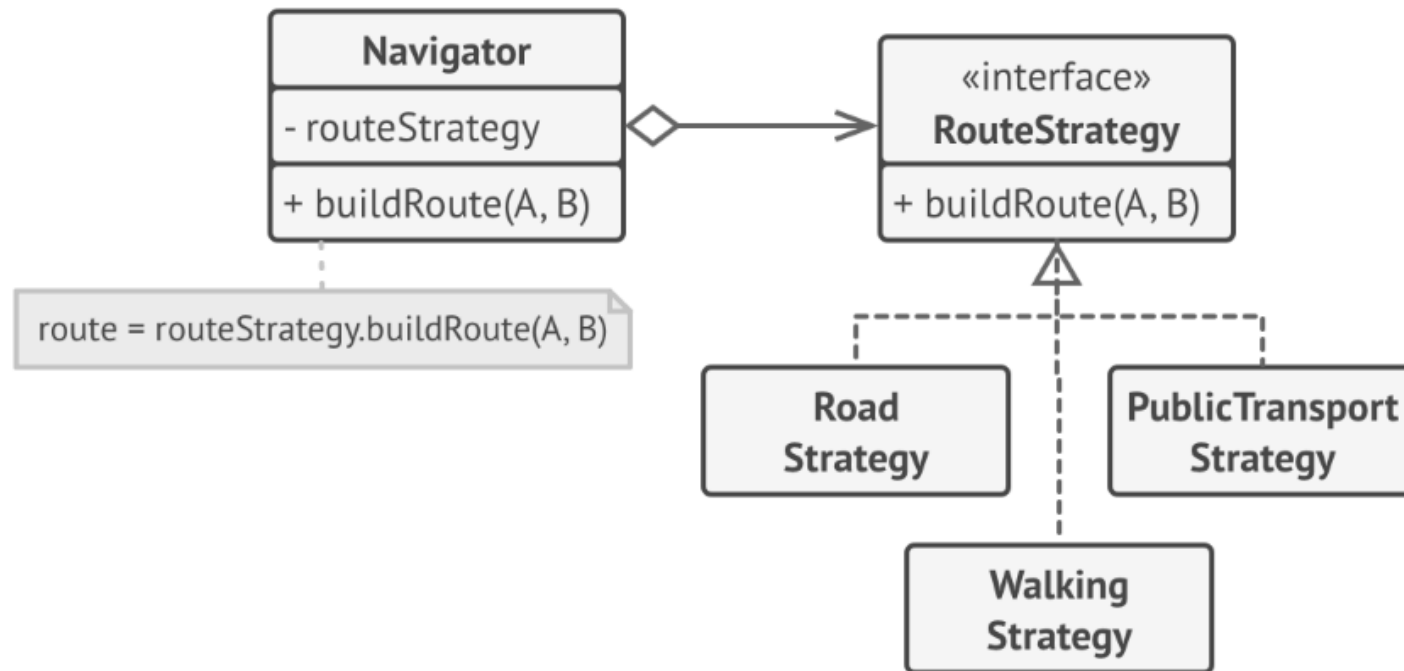
👉 A "solução" 3 implica criar uma hierarquia de classes, onde as subclasses apenas redefinem um método (*polimorfismo*).

Strategy | Solução Proposta 😊

O padrão **Strategy** sugere pegar numa classe que faz uma **tarefa** específica de muitas formas diferentes e extrair estes "algoritmos" para classes separadas chamadas de *estratégias*.

- A classe original (**context**) guarda uma referência para uma das estratégias - irá *delegar* o trabalho da tarefa para este objeto.
 - O *context* não é responsável por escolher a estratégia apropriada à tarefa. O **cliente** passar-lhe-á a estratégia desejada.
- O *context* não conhece os detalhes das estratégias. Trabalha com todas elas através de uma *interface* comum genérica (**strategy**) que expõe um método para executar o algoritmo encapsulado na estratégia selecionada (**concrete strategy**).

Strategy | Solução Proposta 😊

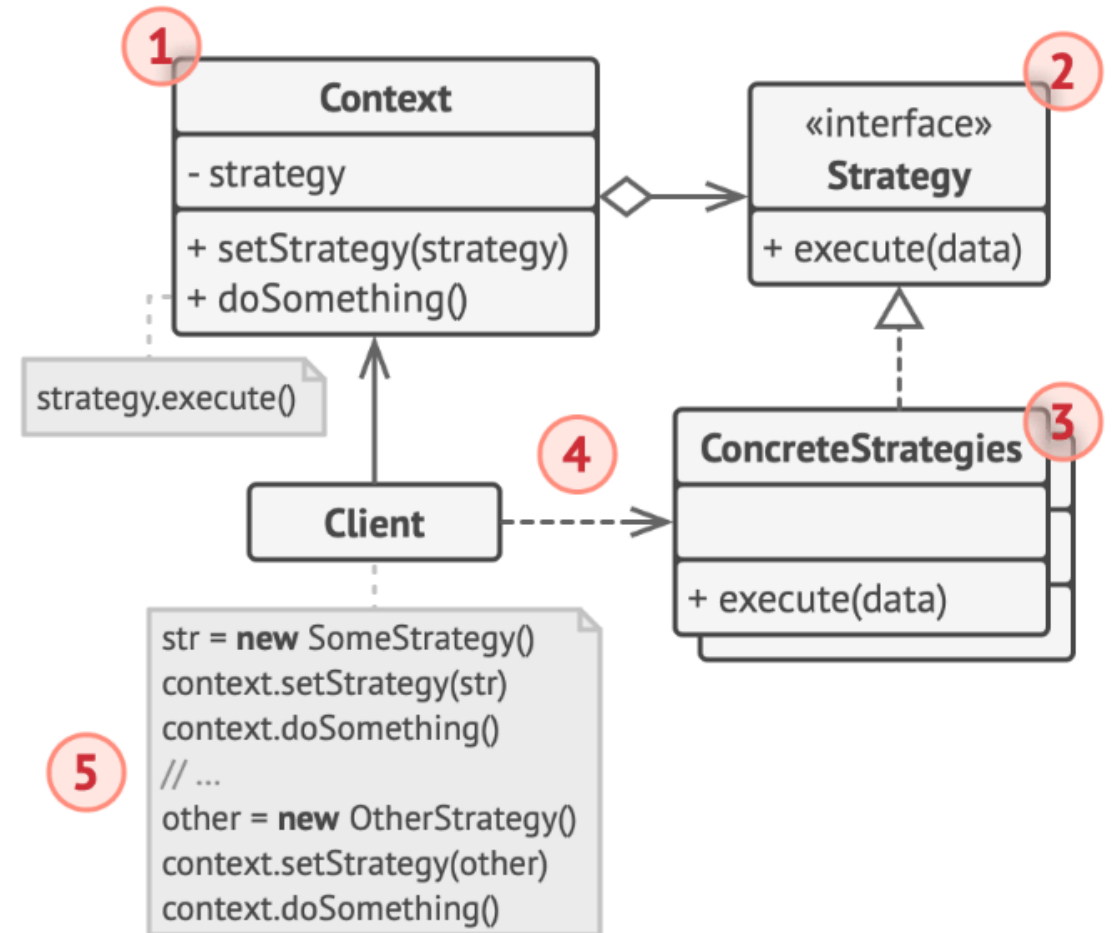


👍 O elemento *context* torna-se independente das estratégias concretas, logo podemos adicionar novas estratégias ao longo do tempo sem ter de modificar o código do *context*.

👍 Podemos alterar o "comportamento" de **Navigator** em *tempo-de-execução*.

Strategy | Participantes

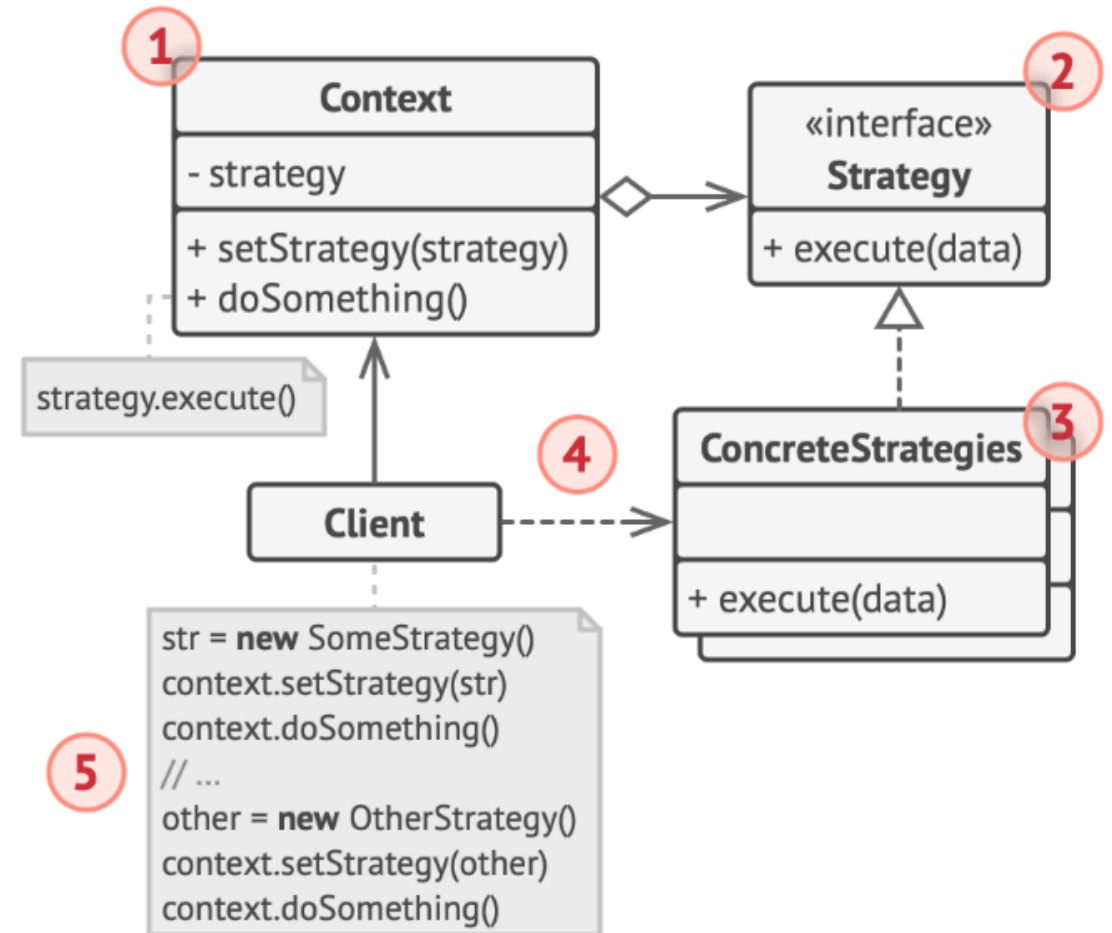
1. **Context**: mantém uma referência para estratégia (para a qual irá delegar a tarefa) e comunica através da interface;
2. **Strategy**: *interface* comum de todas as estratégias. Contém o(s) método(s) utilizados para executar a estratégia;
3. **Concrete Strategies**: implementação real das diferentes estratégias (algoritmos);
4. **Client**: Cria a estratégia desejada, passa-a ao *context* (`setStrategy`) e invoca a operação no *context*.



Strategy | Participantes

! Nota

- Uma vez que as *concrete strategies* são classes separadas, o método `execute` tem de receber nos parâmetros toda a informação necessária para desempenhar a tarefa.
- A interface tem de prever isto.



Strategy | Exemplo de Aplicação

Ficha de Aluno 🎓

Repositório de apoio à aula:

- https://github.com/estsetubal-pa/JavaPatterns_Strategy
-

Considere este programa que modela a ficha de um aluno através da classe `StudentRecord`, i.e., o seu percurso académico - unidades curriculares efetuadas e notas obtidas.

É possível obter a média aritmética das notas já obtidas.

Strategy | Exemplo de Aplicação

Ficha de Aluno 🎓

Pretende-se poder calcular a média do aluno de diferentes formas:

- Média aritmética;
- Média ponderada relativamente aos ECTS de cada UC.

Para tal aplica-se o padrão `**strategy**` para resolver este novo requisito.

Strategy | Exemplo de Aplicação

Ficha de Aluno 🎓 SOLUÇÃO

Mapeamento dos Participantes no padrão

1. **Context** - `StudentRecord`

2. **Strategy** - interface `Strategy` : `double calculateAverage(Map<Course, Integer> r)`

3. **Concrete Strategy** `StrategyArithmetic` e `StrategyWeighted`

Strategy | Exemplo de Aplicação: Strategy & ConcreteStrategy

```
public interface Strategy {  
    double calculateAverage(Map<Course, Integer> record);  
}
```

```
public class StrategyArithmetic implements Strategy {  
    @Override  
    public double calculateAverage(Map<Course, Integer> record) {  
        double sum = 0;  
        for (Integer grade : record.values()) {  
            sum += grade;  
        }  
        return sum / record.size();  
    }  
}
```

Strategy | Exemplo de Aplicação: Concrete Strategy

```
public class StrategyWeighted implements Strategy{
    @Override
    public double calculateAverage(Map<Course, Integer> record) {
        double total=0.0, sum=0.0;
        for(Course c: record.keySet()){
            total+=c.getEcts();
            double grade= record.get(c);
            sum+=grade*c.getEcts();
        }
        return sum/total;
    }
}
```

Strategy | Exemplo de Aplicação: Context

```
public class StudentRecord {
    private String studentId;
    private String studentName;
    Strategy strategy;

    public StudentRecord(String studentId, String studentName, Strategy strategy) {
        this.strategy = strategy;
        // ...initialize other fields
    }
    // method to change strategy in runtime
    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }
    // delegate to strategy the way to compute average
    public double computeAverage() {
        return strategy.calculateAverage(record);
    }
}
```

Strategy | Exemplo de Aplicação: Client

```
public class Main {  
    public static void main(String[] args) {  
        StudentRecord record = new StudentRecord("2018", "João Meireles", new StrategyArithmetic());  
        try {  
            record.importFromFile("record.csv");  
        } catch (FileNotFoundException e) {  
            System.err.println(e.getMessage());  
        }  
        /* Compute average */  
        double average = record.computeAverage();  
        System.out.println(String.format("Course average: %.2f", average));  
        //change Strategy  
        record.setStrategy(new StrategyWeight);  
        average = record.computeAverage();  
        System.out.println(String.format("Course average: %.2f", average));  
    }  
}
```


Strategy | Pros e Contras

- ✓ Podemos alterar a estratégia em tempo-de-execução;
- ✓ Podemos substituir herança por composição;
- ✓ *Single Responsibility Principle*. Cada estratégia concreta é responsável por um único algoritmo;
- ✓ *Open/Closed Principle*. É fácil adicionar novas estratégias sem alterar o *context*;
- ✗ Se existir um número muito baixo e fixo de estratégias, pode não compensar a criação de novas interfaces e estratégias decorrentes do padrão;
- ✗ O *cliente* tem de conhecer as estratégias existentes para poder selecionar uma.

Strategy | Webgrafia

- <https://refactoring.guru/design-patterns/strategy>

As imagens dos slides 4, 6, 8, 9 e 10 são creditadas a este *website*.

- What is the Strategy Pattern? (Software Design Patterns)

<https://www.youtube.com/watch?v=9uDFHTWCKkQ>